



传感器及应用系统课程设计说明书

数字霍尔电流表设计



主要内容、基本要求、主要参考资料等：

一、主要内容：

- (1) 整体电路设计（画出电路组成框图）；
- (2) 信号采样电路设计；
- (3) 信号放大电路设计，电路参数选取、数据计算；
- (4) 信号滤波电路设计，电路参数选取、数据计算；

二、基本本要求：

- (1) 选用合适的压电传感器组成测量电路
- (2) 电路组成：压电式传感器、运算放大电路、滤波电路、显示电路；
- (3) 假设在实验装置上进行模拟实验，测量出脉搏次数与输出信号之间的关系；
- (4) 写出 5000 字左右的工作原理说明，附系统图一张。

三、主要参考资料：



数字霍尔电流表系统设计

电子信息工程 0

摘要: 详细介绍新型线性电流传感器 ACS712 的特点, 工作原理, 特性曲线及典型应用电路, 对 ACS712 与 STC12C2052AD 单片机的接口进行了分析和设计。设计了一种基于 ACS712 的直流检测系统。ACS712 使用方便、性价比高、绝缘电压高等特点, 主要应用于电动机控制、载荷检测和管理、开关式电源和过电流故障保护等, 特别是那些要求电气绝缘却未使用光电绝缘器或其它昂贵绝缘技术的应用中。它能准确, 实时地检测电流, 使设备隐患得到及时的处理, 确保人身安全和设备安全。

关键词: ACS712; 特性曲线; STC12C2052AD; 电流检测; RS-232; 74HC595



目录

1 概述	3
2 霍尔电流计系统设计方案	3
3 霍尔电流计系统硬件电路设计	4
3.1 传感器采集放大电路.....	4
3.1.1 ACS712 内部结构及工作原理	4
3.1.2 特性曲线.....	5
3.2 ACS712 与 AD 接口	6
3.3 STC12C2052AD 单片机 AD 采集电路.....	6
3.4 数码管显示电路	10
3.5 电源稳压电路.....	11
3.6 下载通信电路.....	12
3.7 系统程序.....	13
4 总结	13
参考文献.....	15
系统总图	16
程序	17

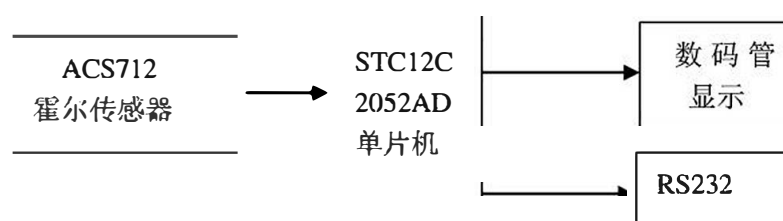


1 概述

在工业、汽车、商业和通信系统中，为了确保设备安全和人身安全，经常需要对设备的某些关键点进行电流检测，传统的检测方法存在测量精度不高，反应时间长，对于大电流一般采用电流互感器输出端不能开路，突发性绝缘击穿等缺点。新型线性电流传感器 ACS712 能有效克服这些缺点，为工业、汽车、商业和通信系统中的交流或直流电流检测提供经实惠的精密解决方案。

2 霍尔电流计系统设计方案

本系统采用以 STC 系列的 STC12C2052AD 单片机系统为核心开发霍尔电流计系统。系统硬件原理框图如图 1-1：



图：1-1

系统硬件原理图 1-1 由 ACS712 传感器，STC12C2052AD 单片机自带 AD 模数转换，74HC595 数码管显示和 RS232 串口通信等构成。整个系统由微处理器控制，根据霍尔原理设计的。霍尔元件之作用原理也就是霍尔效应，所谓霍尔效应指将电流 I 通至一物质，并对与电流成正角之方向施加磁场 B 时，在电流与磁场两者之直角方向所产生的电位差 V 之现象。ACS712 在一定的电流范围内输出电压和电流成线性关系，内部自带放大，滤波电路，通过单片机自带 8 位 AD 采集数据，再由软件滤波补偿，用数码管实时显示当前电流值它的各部分电路的说明如下。

(1) ACS712 采集电流，放大输出电压信号，经滤波电路后电流和电压建立线性关系

(2) STC12C2052AD 单片机通过内部 8 位 AD 实时检测电压，通过线性关系和补偿建立电流和电压的关系

(3) 通过显示电路将测距结果实时地显示出来。



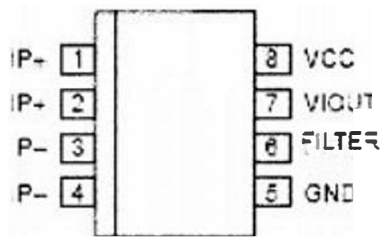
(4) 通过串口 RS232 和上位机通信，实时传送数据。

3 霍尔电流计系统硬件电路设计

系统由三大部分组成：(1) 传感器采集放大电路；(2) STC12C2052AD 单片机 AD 采集电路；(3) 数码管显示电路 (4) 电源稳压电路 (5) 下载通信电路；

3.1 传感器采集放大电路

ACS712是一种线性电流传感器，该器件内置有精确的低偏置的线性霍尔传感器电路，能输出与检测的交流或直流电流成比例的电压。具有低噪声，响应时间快（对应步进输入电流，输出上升时间为5 S），50千赫带宽，总输出误差最大为4%，高输出灵敏度（66 mV / A~185mV / A），ACS712采用小型的SOIC8 封装，采用单电源5V 供电。引脚 1和2、3和4均内置有保险，为待测电流的两个输入端，当检测直流电流时，1 和2、3和4分别为待测电流的输入端和输出端。



图：3-1

3.1.1 ACS712 内部结构及工作原理

ACS712器件主要由靠近芯片表面的铜制的电流通路和精确的低偏置线性霍尔传感器电路等组成。被测电流流经的通路(引脚1和2，3和4之间的电路)的内电阻通常是1.2毫欧，具有较低的功耗。被测电流通路与传感器引脚(引脚5~8)压>2.1kVRMS，几乎是绝缘的。流经铜制电流通路的电流所产生的磁场，能够被片内的霍尔IC感应并将其转化为成比例的电压。通过将磁性信号尽量靠近霍尔传 感器来实现器件精确度的最优化。精确的成比例的输出电压由稳定斩波型低偏置BiCMOS霍尔集成电路提供，该集成电路在出厂时已进行了精确的编程。稳定斩波技术是一种新技术，它给片内的霍尔元器件和放大器提供最小的偏置电压，该技术几乎可以消除芯片由于温度所产生的输出。

引脚	名称	功能描述
1 和 2	IP+	被测电流输入或输出
3 和 4	IP-	被测电流输入或输出
5	GND	信号地
6	FILTER	外接电容
7	VIOUT	模拟电压输出
8	VCC	电源电压

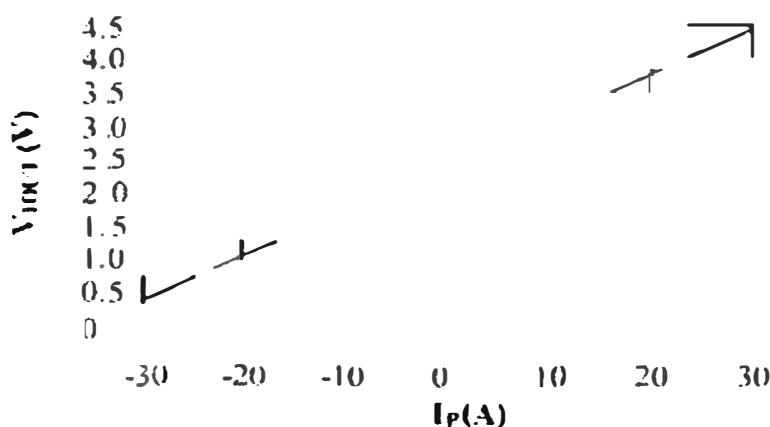


图：3-1-1

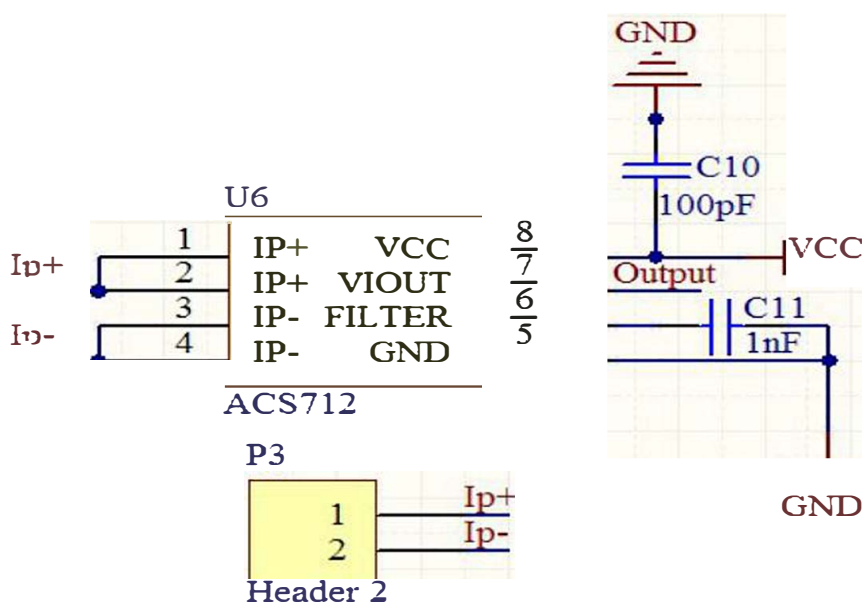
ACS712内含一个电阻 R_F (INT) [I个缓冲放大器，用户可以通过FITER引脚(第6脚)外接一个容 C_F 与 R_F (INT)组成一个简单的外接RC低通滤波器。

3. 1. 2 特性曲线

ACS712的30A输出电压与检测电流关系的特性曲线，在检测范围 $\pm 30A$ 内，传感器的输出电压和检测电流成正比，几乎不受温度的影响。图图：3-1-2为ACS712-30A检测灵敏度与电流关系的特性曲线，输出灵敏度约为66mV/A，受温度的影响很小输出电压和检测电流的关系



图：3-1-2



霍尔传感器



图： 3-1-3

3.2 ACS712 与 AD 接口

系统采用ACST12EICTR-30A-T进行电流检测，ACS712ELCTR-30A-T的输出端VOUT接到STC12C2052AD的P1.0进行模数转换，ACS712的VCC=5V，根据图3-1-3，ACS712的电压输出Vout和被检测的电流IP间的关系为： $V_{OUT}=(2 / 30) I_p +2.5$ 。由图3-1-2知：V 的输出范围为0.5V~4.5V，量程为4V，若采用8位的ADC 时，量化单位为 $A=4/256V=15.6\text{mv}$ 小于ACS712的输出灵敏度，即用8位的ADC在转换精度上可以满足需要. 设计中根据STC12C2052AD的AD特性, AD输入的模拟量VIN和输出的数字量D之间的关系为： $D=256 \times V_{in} / V_{cc}$ ，另外 $V_{out} = V_{in}$ 所以STC12C2052AD的AD输出的数字量D和被检测电流IP间有如下的 关系： $D=(128 \times I_p) / 30+128$ 。即被检测电流与A / D转换后的数字量间建立了一一对应的关系，当被检测的电流为-30A时，D=0；当被检测的电流为0A时，D=128；当被检测的电流为30A时，D=256，被检测电流的大小通过ACS712和STC12C2052AD的AD转化为数字量后输入到单片机进行处理。

3.3 STC12C2052AD 单片机 AD 采集电路

STC12C2052AD系列单片机的单时钟/机器周期（1T）的单片机，是高速/低功耗/超强抗干扰的新一代8051单片机，指令代码完全兼容传统8051，但速度快8—12倍，STC12C2052AD系列有2路PWM，8路高速8位A/D转换通用I/O（27/23/15个），复位后为：准双向口/弱上拉（普通8051传统I/O口）。可设置成四种模式：准双向口/弱上拉，推挽/强上拉，仅为输入/高阻，开漏。每个I/O口驱动能力均可达到20mA，但整个芯片最大不得超过55MA. STC12C2052AD系列有两路PWM，也可用来当4路D/A使用，也可用来再实现4个定时器。STC12C2052AD系列是8位精度的A/D，速度均可达到100KHZ（10万次/秒）。8路电压输入型A/D，可做温度检测、电池电压检测、按键扫描、频谱检测等。上电复位后P1口为弱上拉型I/O口，用户可以通过软件设置将8路中的任何一路设置为A/D转换，不需作为A/D使用的口可继续作为I/O口作用。需作为A/D使用的口需先将其设置为开漏模式或高阻输入，在P1M0、P1M1 寄存器中对相应的位进行设置。

表2-1 P1M1 P1M0寄存器模式设置

P1M0[7:0] 地址：91h	P1M1[7:0] 地址：92h	I/O口模式（P1.X 如做A/D使用，需先将其设置成开漏或高阻输入）
		准双向口（传统8051 I/O口模式）



0	0	灌电流可达20MA，拉电流为230UA
0	1	推挽输出（强上拉输出，可达20MA，尽量少用）
1	0	仅为输入（高阻），如果该I/O口需作为A/D使用，可选此模式
1	1	开漏（OPEN DRAIN），如果该I/O口需作为A/D使用，可选此模式

表2-2 AD转换寄存器

Mnemonic	Add	Name	7	6	5	4	3	2	1	0	Reset Value
ADC_CONTR	C5h	A/D转换控制寄存器	ADC_POWER	SPE_ED1	SPE_ED0	ADC_FLAG	ADC_START	CHS2	CHS1	CHS0	0XX00000
ADC_DATA	C6h	A/D转换结果寄存器，高8位	_____	_____	_____	_____	_____	_____	_____	_____	XXXXXXXX
ADC_LOW2	Beh	A/D转换结果寄存器，									



		低2位									
--	--	-----	--	--	--	--	--	--	--	--	--

表2- 3ADC_CONTR 特殊功能寄存器：A/D转换控制特殊功能寄存器

A/D转 换控 制寄 存器	ADC_ POWER	SPEED1	SPEED0	ADC _FLAG	ADC _START	CHS2	CHS1	CHS0	0XX0 0000
------------------------	---------------	--------	--------	--------------	---------------	------	------	------	------------------

表2- 4CHS2/CHS1/CHS0:模拟输入通道选择，CHS2/CHS1/CHS0

CHS2	CHS1	CHS0	Analog Channel Select 模拟输入通道选择
0	0	0	选择 P1.0 作为 A/D输入来用
0	0	1	选择 P1.1 作为 A/D输入来用
0	1	0	选择 P1.2 作为 A/D输入来用
0	1	1	选择 P1.3 作为 A/D输入来用
1	0	0	选择 P1.4 作为 A/D输入来用
1	0	1	选择 P1.5 作为 A/D输入来用
1	1	0	选择 P1.6 作为 A/D输入来用
1	1	1	选择 P1.7 作为 A/D输入来用

ADC_START:模数转换器（ADC）转换启动控制位，设置“1”时，开始转换，转换结束后为0。

ADC_FLAG:模数转换器转换结束标志位，当A/D转换完成后，ADC_FLAG=1, 要由软件清0。

表2- 5SPEED1，SPEED：模数转换器转换速度控制位



SPEED1	SPEED0	A/D转换所需时间
1	1	210个时钟周期转换一次，CPU工作频率20MHZ A/D转换速度约100KHZ
1	0	420个时钟周期转换一次
0	1	630个时钟周期转换一次
0	0	840个时钟周期转换一次

ADC_POWER: ADC电源控制 0: 关闭ADC电源; 1: 打开A/D转换器电源, 建议进入空闲模式前, 将ADC电源关闭, ADC_POWER=0。启动A/D转换前一定要确认AD电源已打开, AD转换结束后关闭AD电源可降低功耗, 也可不关闭建议启动A/D转换后, 在A/D转换结束之前, 比改变任何I/O口的状态, 有利于高精度A/D转换

表2-6 ADC_DATA/ADC_LOW2 特殊功能寄存器: A/D转换结果特殊功能寄存器

ADC_DATA	C6h	A/D转换结果寄存器, 全部8位有效, 为10位A/D转换结果的高8位	—	—	—	—	—	—	—	XXXX XXXX
ADC_LOW2	BEh	A/D转换结果寄存器, 只有低2位有效, 为10位A/D转换结果的低2位	X	X	X	X	X	X	— —	XXXX XXXX

模拟/数字转换结果计算公式如下: 结果 (ADC_DATA[7:0], ADC_LOW2[1:0])



$=1024 \times V_{in} / V_{cc}$ V_{in} 为模拟输入通道输入电压, V_{cc} 为单片机实际工作电压, 用单片机工作电压作为模拟参考电压。取ADC_DATA的8位为ADC转换的高8位, 取ADC_LOW2的低2位为ADC转换的低2位, 则为10位精度。如果舍弃ADC_LOW2的低2位, 只用ADC_DATA寄存器的8位则A/D转换结果为8位精度。结果 $ADC_DATA[7:0]=256 \times V_{in} / V_{cc}$ STC12C5410AD系列单片机的参考电压源是输入工作电压VCC, 所以一般不用外接参考电压源。如7805的输出电压是5V, 但实际电压可能是4.88V到4.96V, 用户需要精度比较高的话, 可在出厂时将实际测出的工作电压值记录在单片机内部的EEPROM里面, 以供计算。在本设计中, 考虑现实条件, 我们使用了LM2940作为电压源, 给单片机提供5V电压。

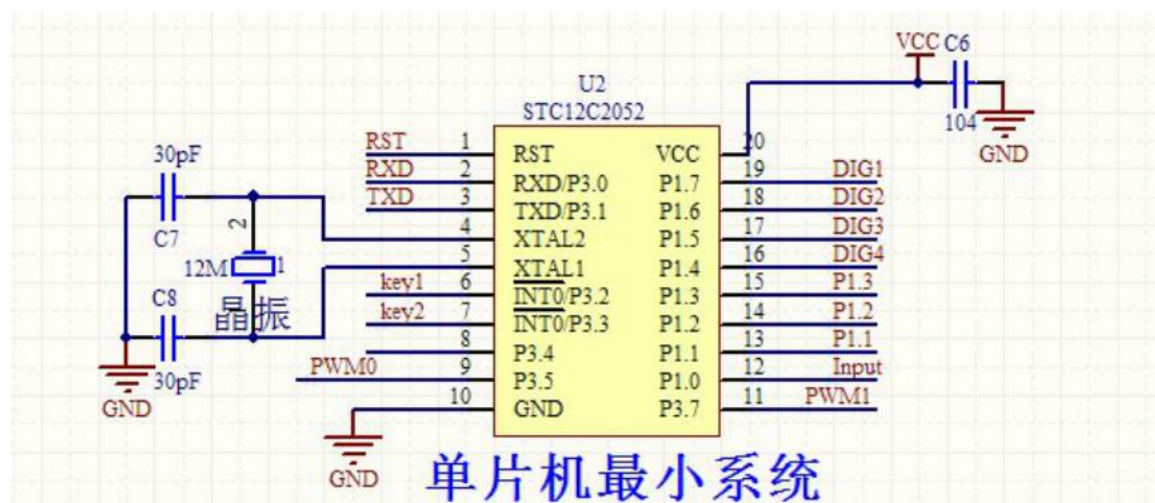


图 3-3

3.4 数码管显示电路

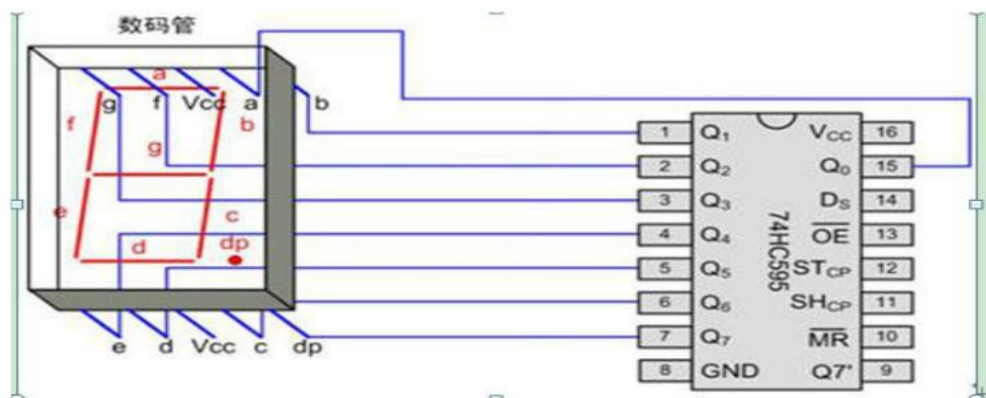
74HC595是具有8位移位寄存器和一个存储器, 三态输出功能。移位寄存器和存储器是分别的时钟。数据在SHCP的上升沿输入, 在STCP的上升沿进入到存储寄存器中去。如果两个时钟连在一起, 则移位寄存器总是比存储寄存器早一个脉冲。移位寄存器有一个串行移位输入(DS), 和一个串行输出(Q7'), 和一个异步的低电平复位(MR), 存储寄存器有一个并行8位的, 具备三态的总线输出, 当使能OE时(为低电平), 存储寄存器的数据输出到总线。

74595的控制端说明: /SCLR(10脚): 低点平时将移位寄存器的数据清零。通常我将它接Vcc。SCK(11脚): 上升沿时数据寄存器的数据移位QA→QB→QC→...→QH; 下降沿移位寄存器数据不变。(脉冲宽度: 5V时, 大于几十纳秒就行了。RCK(12脚): 上升沿时移位寄存器的数据进入数据存储寄存器, 下降沿时存储寄存器数据不变。通常我将RCK置为低点平, 当移位结束后, 在RCK端产生一个正脉冲(5V时, 大于几十纳秒就行了。我通常都选微秒级), 更新显示数据。/G(13脚): 高电平时禁止输出

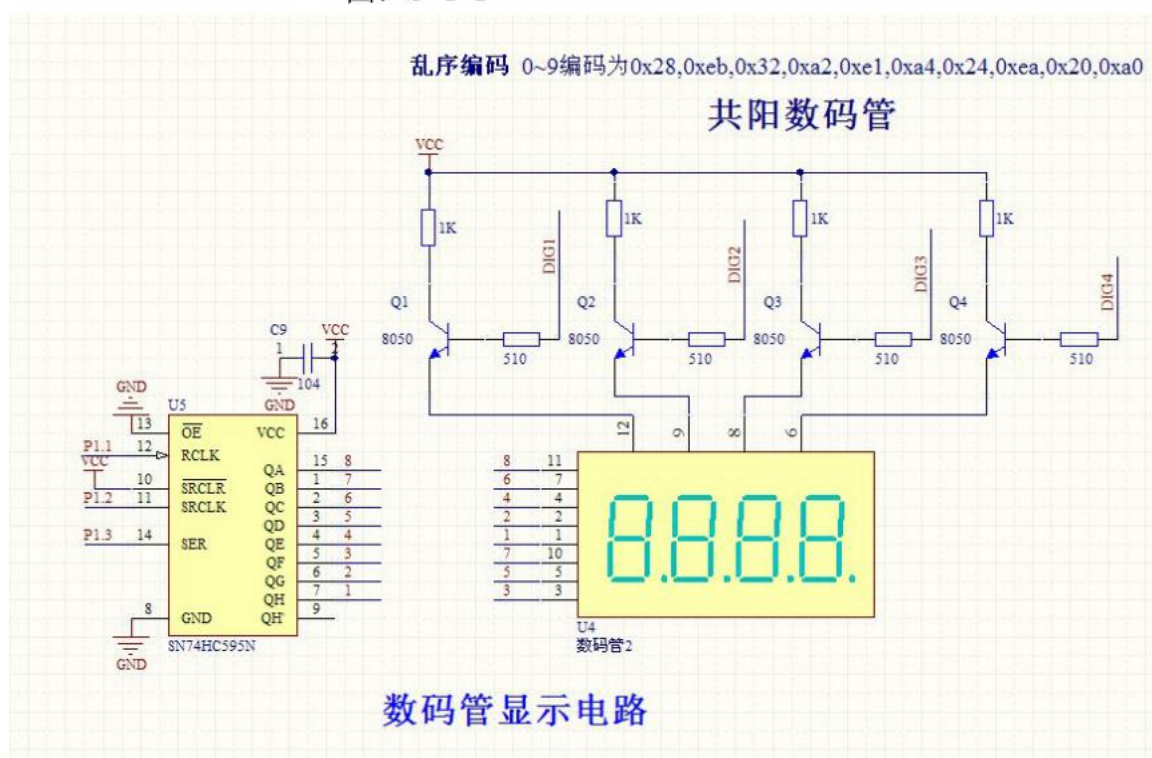


(高阻态)。如果单片机的引脚不紧张,用一个引脚控制它,可以方便地产生闪烁和熄灭效果。比通过数据端移位控制要省时省力。

分析原理图,需要仔细考虑元器件的布局与走线,合理的布局会减少走线的难度。本设计为了方便走线,数码管和74HC595采用自由连接方式,而没有采用顺序连接方式。这大大降低了走线难度。

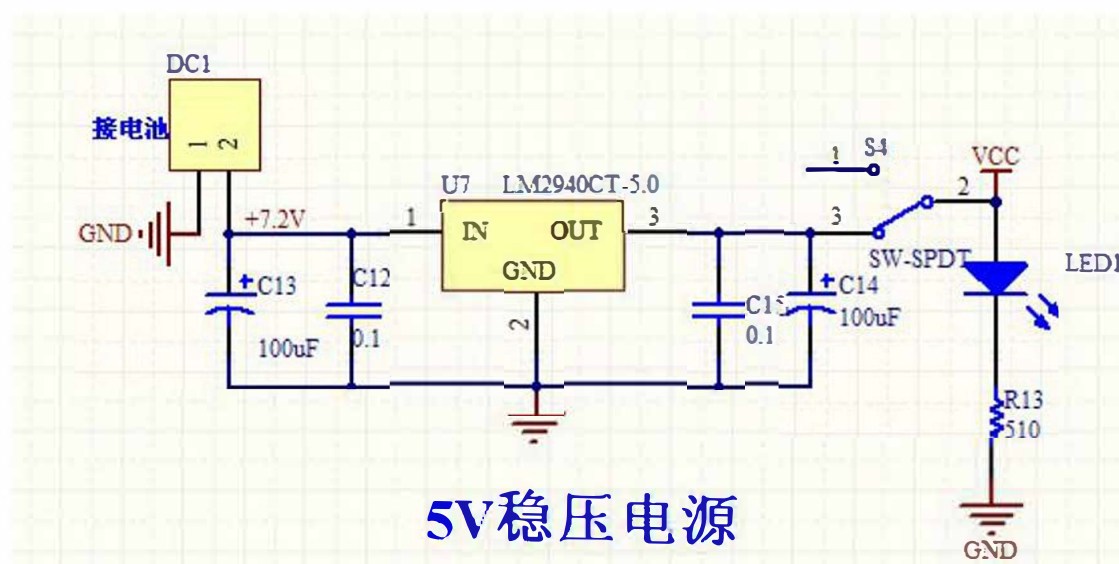


图：3-4-1



3.5 电源稳压电路

LM2940 具有纹波小、电路结构简单的优点,但是效率较低,功耗大,由于 LM2940 的稳压的线性度非常好,所以选用 LM2940-5 单独对其进行供电;



3.6 下载通信电路

MAX232 芯片是美信 (MAXIM) 公司专为 RS-232 标准串口设计的单电源电平转换芯片, 使用+5v 单电源供电。

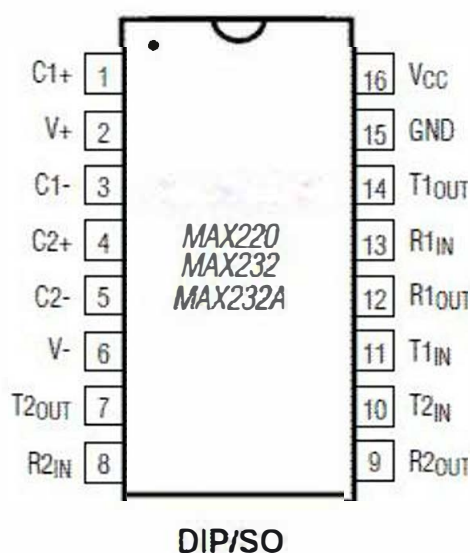


图: 3-6-1

第一部分是电荷泵电路。由 1、2、3、4、5、6 脚和 4 只电容构成。功能是产生 +12v 和 -12v 两个电源, 提供给 RS-232 串口电平的需要。

第二部分是数据转换通道。由 7、8、9、10、11、12、13、14 脚构成两个数据通道。其中 13 脚 (R1IN)、12 脚 (R1OUT)、11 脚 (T1IN)、14 脚 (T1OUT) 为第一数据通道。8 脚 (R2IN)、9 脚 (R2OUT)、10 脚 (T2IN)、7 脚 (T2OUT) 为第二数据通道。TTL/CMOS 数据从 T1IN、T2IN 输入转换成 RS-232 数据从 T1OUT、T2OUT 送到电脑 DB9 插头; DB9 插头的 RS-232 数据从 R1IN、R2IN 输入转换成 TTL/CMOS 数据后从 R1OUT、R2OUT 输出。

第三部分是供电。15 脚 GND、16 脚 VCC (+5v)。

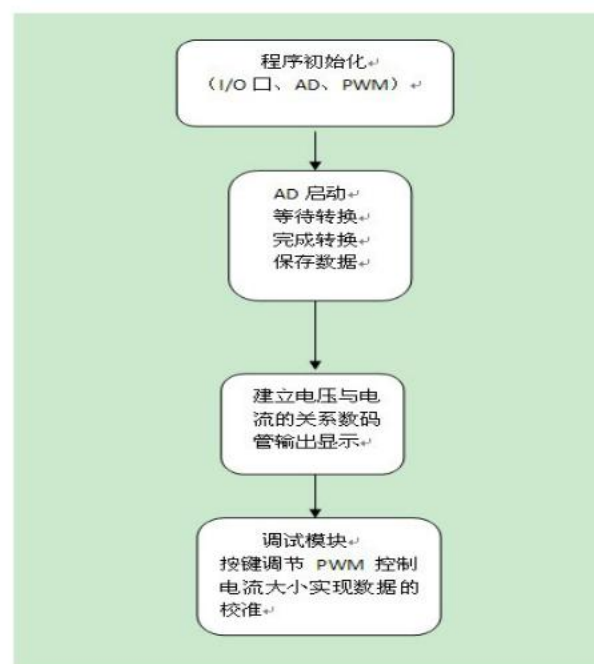
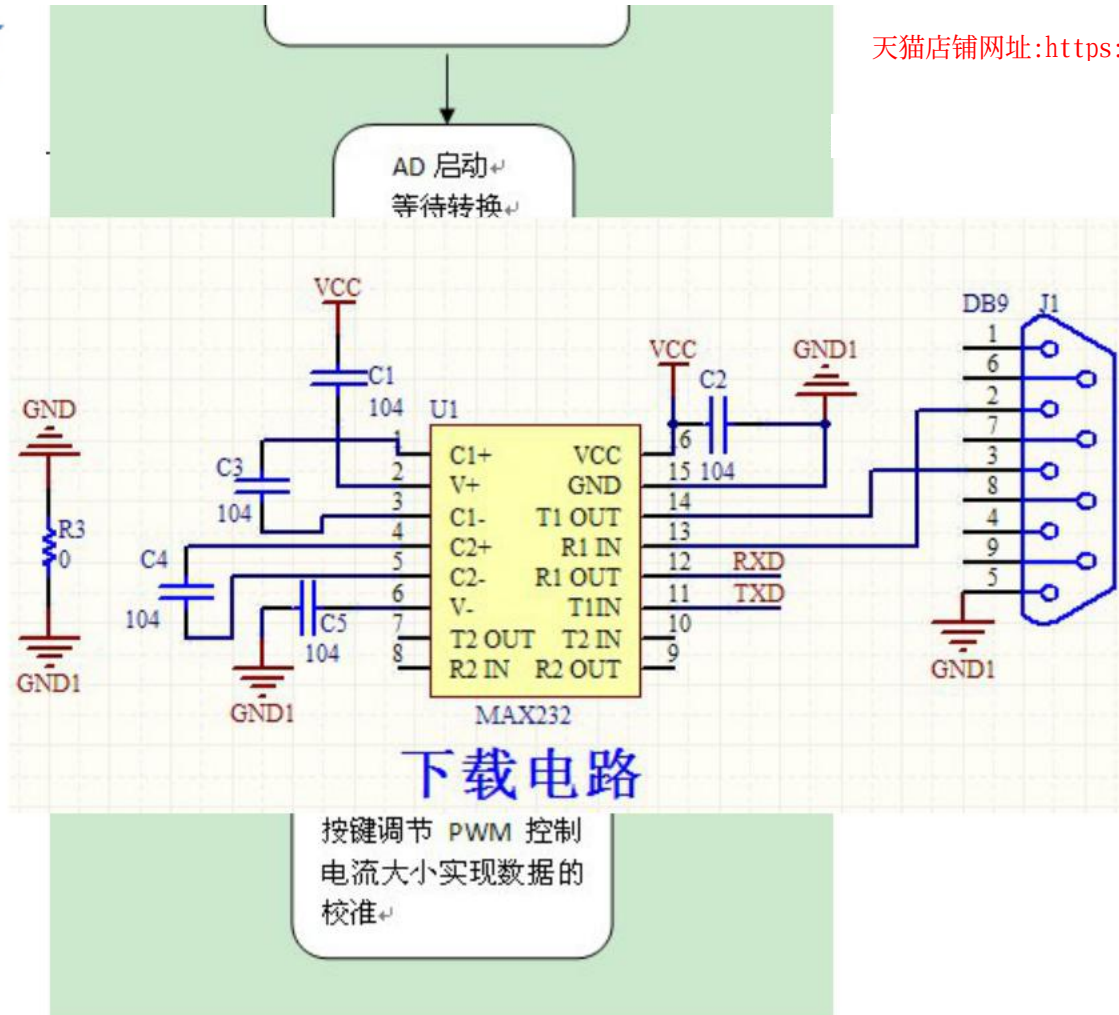
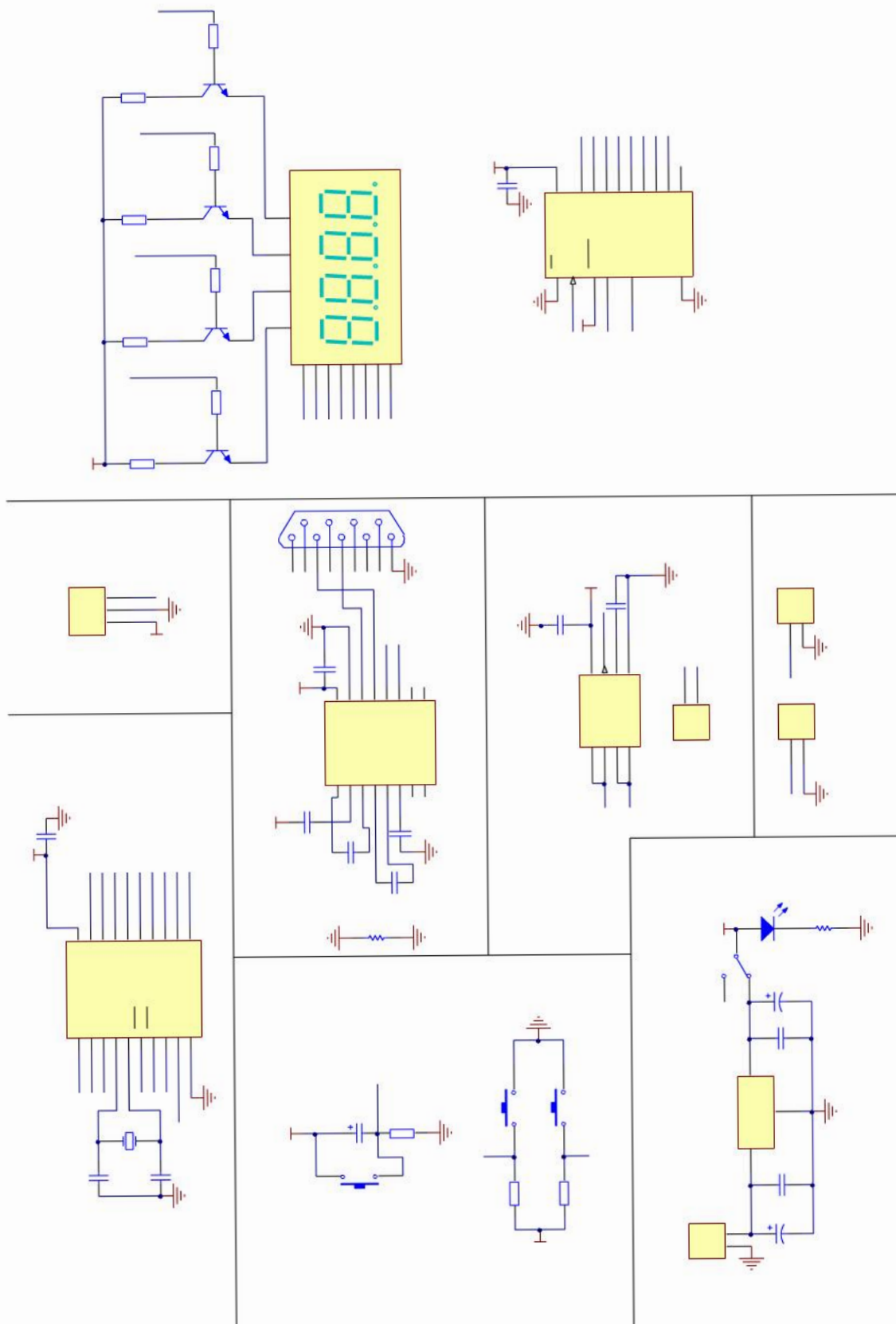


图 3-7 主程序实现流程

4 总结

我设计的课题是霍尔电流计，经过一个星期的思考和准备，通过对课题的设计，大大提高了自己各方面的能力，例如对书本基础知识的掌握程度，对资料的查找方法，对自己知识面的扩展都不得有相对的提高，在学习的同时，我

系统总图:





```
程序: #include <STC12C2052AD.H>

#include <intrins.h>

#define uchar unsigned char

#define uint unsigned int

sbit srclk = P1 ^ 2; //11 数据时钟输入

sbit ser = P1 ^ 3; //14 数据输入

sbit rclk = P1 ^ 1; //12 数据锁存

sbit key1 = P3 ^ 2;

sbit key2 = P3 ^ 3;

uchar code tabnum0[] = { 0x28, 0xeb, 0x32, 0xa2, 0xe1, 0xa4, 0x24, 0xea, 0x20, 0xa0,
0x60, 0xdf }; //字符 共阳 0x60 A 0xdf 小数点

uchar code tabnum1[] = { 0x08, 0xcb, 0x12, 0x82, 0xc1, 0x84, 0x04, 0xca, 0x00, 0xc0,
0x40 };

uchar code weatab[] = { 0x7f, 0xbf, 0xdf, 0xef };

void delaynms (uint xms);

void shuju (uchar date);

void IO_init ();

void AD_init ();

uchar AD ();

void PWM_init();

void PWM_init()//初始化 PWM 频率为 23.4K
{
    CMOD = 0x02; //0000 0010 频率是晶振 / 2 禁止 PCA 定时器溢出中断
    CCAPM1 = 0x42; //0100 0110 输出比较使能 高速输出 PWM 模式
    CL = 0x00;
    CH = 0x00;
    PCA_PWM1 = 0x00; //EPC0H : EPC0L = 00
    CCAP1L = 0xff; //控制输出占空比
    CCAP1H = 0xff; //占空比为 0
    CR = 1;
```



```
}  
  
void AD_init ()  
{  
    ADC_CONTR = 0xe0;  
    delaynms(100);  
}  
  
uchar AD ()  
{  
    uchar adresult, flag = 0;  
    ADC_CONTR = 0xe8;  
    while(flag == 0)  
    {  
        flag = ADC_CONTR & 0x10;  
    }  
    adresult = ADC_DATA;  
    ADC_CONTR &= 0x60;  
    return adresult;  
}  
  
void IO_init ()  
{  
    P1M0 = 0x01;  
    P1M1 = 0x01;  
}  
  
void delaynms (uint xms)  
{  
    uint i, j;  
    for( i = xms; i > 0; i-- )  
        for( j = 250; j > 0; j-- );  
}
```



```
}  
  
void shuju (uchar dat)  
{  
    uchar i;  
    rclk = 0;  
    for(i = 0; i < 8; i++)  
    {  
        srclk = 0;  
        if((dat & 0x80) != 0)  
            ser = 1;  
        else  
            ser = 0;  
        dat <<= 1;  
        srclk = 1;  
    }  
    rclk = 1;  
}  
  
void display (uint num)  
{  
    uchar qian, bai, shi, ge;  
    qian = num / 1000;  
    bai = num / 100 % 10;  
    shi = num / 10 % 10;  
    ge = num % 10;  
    P1 = ~weitab[0];  
    shuju(tabnum1[bai]);  
    delaynms(5);  
    P1 = ~weitab[1];  
    shuju(tabnum0[shi]);
```



```
    delaynms(5);
    P1 = ~weitab[2];
    shuju(tabnum0[ge]);
    delaynms(5);
    P1 = ~weitab[3];
    shuju(0x60);//转换对应为电流 A
    delaynms(5);
}
void main ()
{
    uint vout;
    float a;
    uchar k;
    uchar I;
    IO_init();
    AD_init ();
    PWM_init();
    while(1)
    {
        k = AD();
        a = k * 1.9375;//用 vref=2.48, 248*2/256 = 1.9375
        vout = a ;
        I = (vout - 2.5) * 15;
        display(I); //voutvout
        if(!key1)
        {
            delaynms(10);
            if(!key1)
            {
```



```
        while(!key1)
            display(vout);
        if(CCAP1H == 0)
            CCAP1H = 0;
        else
            CCAP1H -= 10;
    }

}

if(!key2)
{
    delaynms(10);
    if(!key2)
    {
        while(!key2)
            display(vout);
        if(CCAP1H >= 250)
            CCAP1H = 250;
        else
            CCAP1H += 10;
    }
}

}
```